Generation as Computation

Generalised Computation in the Set-theoretic Universe and Beyond
(Thesis Proposal)

Desmond Lau

National University of Singapore

November 25, 2025

Outline

Outline

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type.*

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines (BSSMs) [5], born from the study of dynamical systems.
- Infinite-time Turing machines (ITTMs) [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [9] by Yang *et al.*

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines (BSSMs) [5], born from the study of dynamical systems.
- Infinite-time Turing machines (ITTMs) [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [9] by Yang *et al.*

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines (BSSMs) [5], born from the study of dynamical systems.
- Infinite-time Turing machines (ITTMs) [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [9] by Yang *et al.*

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines (BSSMs) [5], born from the study of dynamical systems.
- Infinite-time Turing machines (ITTMs) [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [9] by Yang *et al*.

Outline

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Outline

1. Introduction
   - Background: Infinitary Computation
   - Background: Formulas as Programs
   - Overview
2. Generalised Computation Within V
   - Sequential Algorithms
   - Generalised Sequential Algorithms
   - Relative Computability
3. Generalised Computation Beyond V
   - Degrees of Small Extensions
   - Local Method Definitions

In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

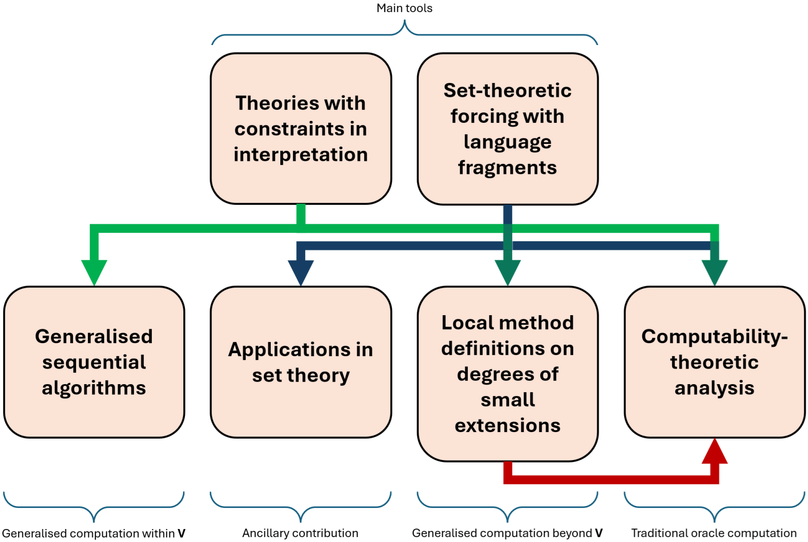
In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

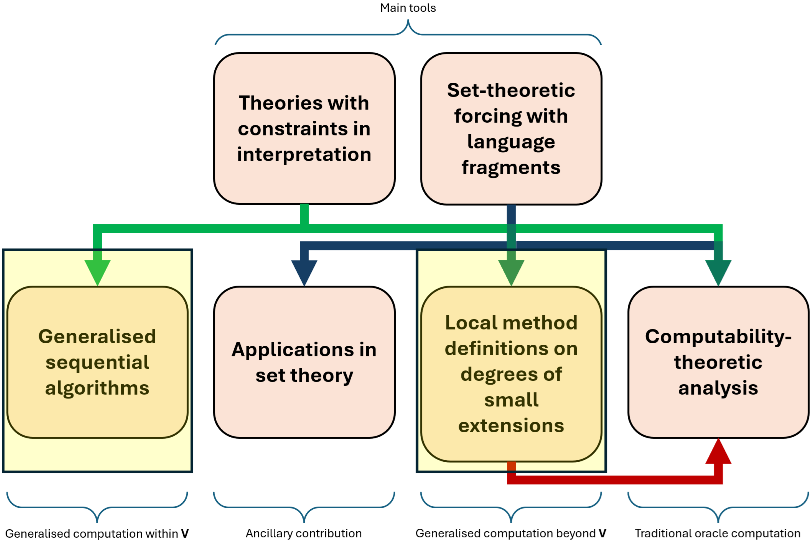
In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

Overview Chart



Today's Focus



Outline

Higher-Level Descriptions

- The explicit models of computation mentioned so far (BSSMs, ITTMs, etc.) are rather low-level and relatively concrete.
- Computer scientists were interested in more inclusive descriptions of algorithms.
- We want to be able to describe programs in pseudocode, instead of being bound to a fixed programming language.
- But what are the confines of pseudocode?
- To formally answer this question, we need to define things at a higher-level of abstraction.

Higher-Level Descriptions

- The explicit models of computation mentioned so far (BSSMs, ITTMs, etc.) are rather low-level and relatively concrete.
- Computer scientists were interested in more inclusive descriptions of algorithms.
- We want to be able to describe programs in pseudocode, instead of being bound to a fixed programming language.
- But what are the confines of pseudocode?
- To formally answer this question, we need to define things at a higher-level of abstraction.

Higher-Level Descriptions

- The explicit models of computation mentioned so far (BSSMs, ITTMs, etc.) are rather low-level and relatively concrete.
- Computer scientists were interested in more inclusive descriptions of algorithms.
- We want to be able to describe programs in pseudocode, instead of being bound to a fixed programming language.
- But what are the confines of pseudocode?
- To formally answer this question, we need to define things at a higher-level of abstraction.

Higher-Level Descriptions

- The explicit models of computation mentioned so far (BSSMs, ITTMs, etc.) are rather low-level and relatively concrete.
- Computer scientists were interested in more inclusive descriptions of algorithms.
- We want to be able to describe programs in pseudocode, instead of being bound to a fixed programming language.
- But what are the confines of pseudocode?
- To formally answer this question, we need to define things at a higher-level of abstraction.

Higher-Level Descriptions

- The explicit models of computation mentioned so far (BSSMs, ITTMs, etc.) are rather low-level and relatively concrete.
- Computer scientists were interested in more inclusive descriptions of algorithms.
- We want to be able to describe programs in pseudocode, instead of being bound to a fixed programming language.
- But what are the confines of pseudocode?
- To formally answer this question, we need to define things at a higher-level of abstraction.

Higher-Level Descriptions

- The explicit models of computation mentioned so far (BSSMs, ITTMs, etc.) are rather low-level and relatively concrete.
- Computer scientists were interested in more inclusive descriptions of algorithms.
- We want to be able to describe programs in pseudocode, instead of being bound to a fixed programming language.
- But what are the confines of pseudocode?
- To formally answer this question, we need to define things at a higher-level of abstraction.

Higher-Level Descriptions

- Moschovakis came up with a theory of recursors in an attempt to answer what it means for two programs to be equivalent.
- Gurevich gave axioms (postulates) that a representation of a program should satisfy.
- Gurevich calls any such representation an *algorithm*.
- His most basic set of axioms classify precisely the *sequential algorithms*.

Higher-Level Descriptions

- Moschovakis came up with a theory of recursors in an attempt to answer what it means for two programs to be equivalent.
- Gurevich gave axioms (postulates) that a representation of a program should satisfy.
- Gurevich calls any such representation an *algorithm*.
- His most basic set of axioms classify precisely the *sequential algorithms*.

Higher-Level Descriptions

- Moschovakis came up with a theory of recursors in an attempt to answer what it means for two programs to be equivalent.
- Gurevich gave axioms (postulates) that a representation of a program should satisfy.
- Gurevich calls any such representation an *algorithm*.
- His most basic set of axioms classify precisely the *sequential algorithms*.

Higher-Level Descriptions

- Moschovakis came up with a theory of recursors in an attempt to answer what it means for two programs to be equivalent.
- Gurevich gave axioms (postulates) that a representation of a program should satisfy.
- Gurevich calls any such representation an *algorithm*.
- His most basic set of axioms classify precisely the *sequential algorithms*.

Higher-Level Descriptions

- Moschovakis came up with a theory of recursors in an attempt to answer what it means for two programs to be equivalent.
- Gurevich gave axioms (postulates) that a representation of a program should satisfy.
- Gurevich calls any such representation an *algorithm*.
- His most basic set of axioms classify precisely the *sequential algorithms*.

Sequential Algorithm: Definition

Gurevich's formulation involves proper classes, but it can be adapted to avoid size issues.

Definition

- A *sequential algorithm* comprises a set of *states*, including *initial states*, and a *transition function* mapping one state to another.
- Every state in a sequential algorithm is a first-order structure with the same base set and finite signature.
- In identifying the next state, the transition function determines changes to the current state based on finite information about the current state (*bounded exploration*).

Sequential Algorithm: Definition

Gurevich's formulation involves proper classes, but it can be adapted to avoid size issues.

Definition

- A *sequential algorithm* comprises a set of *states*, including *initial states*, and a *transition function* mapping one state to another.
- Every state in a sequential algorithm is a first-order structure with the same base set and finite signature.
- In identifying the next state, the transition function determines changes to the current state based on finite information about the current state (*bounded exploration*).

Sequential Algorithm: Definition

Gurevich's formulation involves proper classes, but it can be adapted to avoid size issues.

Definition

- A *sequential algorithm* comprises a set of *states*, including *initial states*, and a *transition function* mapping one state to another.
- Every state in a sequential algorithm is a first-order structure with the same base set and finite signature.
- In identifying the next state, the transition function determines changes to the current state based on finite information about the current state (*bounded exploration*).

Sequential Algorithm: Definition

Gurevich's formulation involves proper classes, but it can be adapted to avoid size issues.

Definition

- A *sequential algorithm* comprises a set of *states*, including *initial states*, and a *transition function* mapping one state to another.
- Every state in a sequential algorithm is a first-order structure with the same base set and finite signature.
- In identifying the next state, the transition function determines changes to the current state based on finite information about the current state (*bounded exploration*).

Sequential Algorithm: Definition

Gurevich's formulation involves proper classes, but it can be adapted to avoid size issues.

Definition

- A *sequential algorithm* comprises a set of *states*, including *initial states*, and a *transition function* mapping one state to another.
- Every state in a sequential algorithm is a first-order structure with the same base set and finite signature.
- In identifying the next state, the transition function determines changes to the current state based on finite information about the current state (*bounded exploration*).

Sequential Algorithm: Definition

Gurevich's formulation involves proper classes, but it can be adapted to avoid size issues.

Definition

- A *sequential algorithm* comprises a set of *states*, including *initial states*, and a *transition function* mapping one state to another.
- Every state in a sequential algorithm is a first-order structure with the same base set and finite signature.
- In identifying the next state, the transition function determines changes to the current state based on finite information about the current state (*bounded exploration*).

Generality and Intuitiveness

- States are analogous to Turing machine configurations.
- An initial state corresponds to a starting configuration upon receiving an input.
- A final state corresponds to an end configuration from which the output can be read.
- A transition function parallels that of a Turing machine: it codes how the machine behaves.
- Incidentally, this general framework of

$$\text{algorithm} = \text{state space} + \text{transition function}$$

is also adopted in recursor theory; Moschovakis calls such a representation an iterator.

Generality and Intuitiveness

- States are analogous to Turing machine configurations.
- An initial state corresponds to a starting configuration upon receiving an input.
- A final state corresponds to an end configuration from which the output can be read.
- A transition function parallels that of a Turing machine: it codes how the machine behaves.
- Incidentally, this general framework of

$$\text{algorithm} = \text{state space} + \text{transition function}$$

is also adopted in recursor theory; Moschovakis calls such a representation an iterator.

Generality and Intuitiveness

- States are analogous to Turing machine configurations.
- An initial state corresponds to a starting configuration upon receiving an input.
- A final state corresponds to an end configuration from which the output can be read.
- A transition function parallels that of a Turing machine: it codes how the machine behaves.
- Incidentally, this general framework of

$$\text{algorithm} = \text{state space} + \text{transition function}$$

is also adopted in recursor theory; Moschovakis calls such a representation an iterator.

Generality and Intuitiveness

- States are analogous to Turing machine configurations.
- An initial state corresponds to a starting configuration upon receiving an input.
- A final state corresponds to an end configuration from which the output can be read.
- A transition function parallels that of a Turing machine: it codes how the machine behaves.
- Incidentally, this general framework of

$$\text{algorithm} = \text{state space} + \text{transition function}$$

is also adopted in recursor theory; Moschovakis calls such a representation an iterator.

Generality and Intuitiveness

- States are analogous to Turing machine configurations.
- An initial state corresponds to a starting configuration upon receiving an input.
- A final state corresponds to an end configuration from which the output can be read.
- A transition function parallels that of a Turing machine: it codes how the machine behaves.
- Incidentally, this general framework of

$$\text{algorithm} = \text{state space} + \text{transition function}$$

is also adopted in recursor theory; Moschovakis calls such a representation an iterator.

Generality and Intuitiveness

- States are analogous to Turing machine configurations.
- An initial state corresponds to a starting configuration upon receiving an input.
- A final state corresponds to an end configuration from which the output can be read.
- A transition function parallels that of a Turing machine: it codes how the machine behaves.
- Incidentally, this general framework of

$$\text{algorithm} = \text{state space} + \text{transition function}$$

is also adopted in recursor theory; Moschovakis calls such a representation an iterator.

To the Infinite

- For an abstract computation model, sequential algorithms are simple to describe and very inclusive.
- It seems like the go-to candidate for generalisation, if we want an flexible and high-level model of infinitary computation.
- Two principles, isolated by Sieg in [8] as integral to classical models of computation, guide our search for the "correct" infinitary version of sequential algorithms.
    - *Boundedness*: "there is a fixed finite bound on the number of configurations a computor can immediately recognise".
    - *Locality*: "a computor can change only immediately recognisable (sub)configurations".

To the Infinite

- For an abstract computation model, sequential algorithms are simple to describe and very inclusive.
- It seems like the go-to candidate for generalisation, if we want an flexible and high-level model of infinitary computation.
- Two principles, isolated by Sieg in [8] as integral to classical models of computation, guide our search for the "correct" infinitary version of sequential algorithms.
  - *Boundedness*: "there is a fixed finite bound on the number of configurations a computor can immediately recognise".
  - *Locality*: "a computor can change only immediately recognisable (sub)configurations".

To the Infinite

- For an abstract computation model, sequential algorithms are simple to describe and very inclusive.
- It seems like the go-to candidate for generalisation, if we want an flexible and high-level model of infinitary computation.
- Two principles, isolated by Sieg in [8] as integral to classical models of computation, guide our search for the "correct" infinitary version of sequential algorithms.
  - *Boundedness*: "there is a fixed finite bound on the number of configurations a computor can immediately recognise".
  - *Locality*: "a computor can change only immediately recognisable (sub)configurations".

To the Infinite

- For an abstract computation model, sequential algorithms are simple to describe and very inclusive.
- It seems like the go-to candidate for generalisation, if we want an flexible and high-level model of infinitary computation.
- Two principles, isolated by Sieg in [8] as integral to classical models of computation, guide our search for the "correct" infinitary version of sequential algorithms.
  - *Boundedness*: "there is a fixed finite bound on the number of configurations a computor can immediately recognise".
  - *Locality*: "a computor can change only immediately recognisable (sub)configurations".

To the Infinite

- For an abstract computation model, sequential algorithms are simple to describe and very inclusive.
- It seems like the go-to candidate for generalisation, if we want an flexible and high-level model of infinitary computation.
- Two principles, isolated by Sieg in [8] as integral to classical models of computation, guide our search for the "correct" infinitary version of sequential algorithms.
  - *Boundedness*: "there is a fixed finite bound on the number of configurations a computor can immediately recognise".
  - *Locality*: "a computor can change only immediately recognisable (sub)configurations".

To the Infinite

- For an abstract computation model, sequential algorithms are simple to describe and very inclusive.
- It seems like the go-to candidate for generalisation, if we want an flexible and high-level model of infinitary computation.
- Two principles, isolated by Sieg in [8] as integral to classical models of computation, guide our search for the "correct" infinitary version of sequential algorithms.
    - *Boundedness*: "there is a fixed finite bound on the number of configurations a computor can immediately recognise".
    - *Locality*: "a computor can change only immediately recognisable (sub)configurations".

Outline

Redefining a Transition Function

- We want to talk about local features between two states.
- Since they are first-order structures with the same signature and base set, local features are synonymous with properties given through the truth predicate $\models$.
- We modify the predicate into $\models_2$ so that we can talk about any two such structures simultaneously.
- Every *generalised sequential algorithm* (*GSeqA*) has its transitions governed by a transition formula: transition from s_1 to s_2 iff $(s_1, s_2) \models_2 \phi$ for transition formula ϕ.

Redefining a Transition Function

- We want to talk about local features between two states.
- Since they are first-order structures with the same signature and base set, local features are synonymous with properties given through the truth predicate $\models$.
- We modify the predicate into $\models_2$ so that we can talk about any two such structures simultaneously.
- Every *generalised sequential algorithm* (*GSeqA*) has its transitions governed by a transition formula: transition from s_1 to s_2 iff $(s_1, s_2) \models_2 \phi$ for transition formula ϕ.

Redefining a Transition Function

- We want to talk about local features between two states.
- Since they are first-order structures with the same signature and base set, local features are synonymous with properties given through the truth predicate $\models$.
- We modify the predicate into $\models_2$ so that we can talk about any two such structures simultaneously.
- Every *generalised sequential algorithm* (*GSeqA*) has its transitions governed by a transition formula: transition from s_1 to s_2 iff $(s_1, s_2) \models_2 \phi$ for transition formula ϕ.

Redefining a Transition Function

- We want to talk about local features between two states.
- Since they are first-order structures with the same signature and base set, local features are synonymous with properties given through the truth predicate $\models$.
- We modify the predicate into $\models_2$ so that we can talk about any two such structures simultaneously.
- Every *generalised sequential algorithm* (*GSeqA*) has its transitions governed by a transition formula: transition from s_1 to s_2 iff $(s_1, s_2) \models_2 \phi$ for transition formula ϕ.

Redefining a Transition Function

- We want to talk about local features between two states.
- Since they are first-order structures with the same signature and base set, local features are synonymous with properties given through the truth predicate $\models$.
- We modify the predicate into $\models_2$ so that we can talk about any two such structures simultaneously.
- Every *generalised sequential algorithm* (*GSeqA*) has its transitions governed by a transition formula: transition from s_1 to s_2 iff $(s_1, s_2) \models_2 \phi$ for transition formula ϕ.

Redefining a Transition Function

- We want to talk about local features between two states.
- Since they are first-order structures with the same signature and base set, local features are synonymous with properties given through the truth predicate $\models$.
- We modify the predicate into $\models_2$ so that we can talk about any two such structures simultaneously.
- Every *generalised sequential algorithm* (*GSeqA*) has its transitions governed by a transition formula: transition from s_1 to s_2 iff $(s_1, s_2) \models_2 \phi$ for transition formula ϕ.

Redefining a Transition Function

- Originally, bounded exploration is defined by quantifying over all states of a sequential algorithm.
- It suffices but is often not easy to verify.
- By restricting transition formulas, we can formalise bounded exploration locally.
- The said restrictions are facilitated by more fundamental design decisions.

Redefining a Transition Function

- Originally, bounded exploration is defined by quantifying over all states of a sequential algorithm.
- It suffices but is often not easy to verify.
- By restricting transition formulas, we can formalise bounded exploration locally.
- The said restrictions are facilitated by more fundamental design decisions.

Redefining a Transition Function

- Originally, bounded exploration is defined by quantifying over all states of a sequential algorithm.
- It suffices but is often not easy to verify.
- By restricting transition formulas, we can formalise bounded exploration locally.
- The said restrictions are facilitated by more fundamental design decisions.

Redefining a Transition Function

- Originally, bounded exploration is defined by quantifying over all states of a sequential algorithm.
- It suffices but is often not easy to verify.
- By restricting transition formulas, we can formalise bounded exploration locally.
- The said restrictions are facilitated by more fundamental design decisions.

Redefining a Transition Function

- Originally, bounded exploration is defined by quantifying over all states of a sequential algorithm.
- It suffices but is often not easy to verify.
- By restricting transition formulas, we can formalise bounded exploration locally.
- The said restrictions are facilitated by more fundamental design decisions.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Base Sets

- To shift the burden of programming to the definition of the transition formula, we streamline the definitions of other components of a GSeqA.
- We specify the base set of any state to be a limit ordinal.
- More precisely, every GSeqA $\mathfrak{M}$ has an associated limit ordinal $\kappa(\mathfrak{M})$.
- Every state of a GSeqA $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in)$ over a fixed signature $\sigma(\mathfrak{M})$ associated with $\mathfrak{M}$.
- This simplification is inspired by the fact that every set can be coded as a set of ordinals.
- The base set being an ordinal also fits the basic intuition of sequential memory when one thinks about computation.

Ordinal Parameters

- For technical purposes, we focus on the subclass of GSeqAs with ordinal parameters, called *generalised sequential algorithm with parameters* (*GSeqAP*s).
- Every GSeqAP $\mathfrak{M}$ has an associated finite set $\mathrm{p}(\mathfrak{M}) \subset \kappa(\mathfrak{M})$.
- Every state of a GSeqAP $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in, \bar{\mathrm{p}}(\mathfrak{M}))$ over $\sigma(\mathfrak{M})$, where $\bar{\mathrm{p}}(\mathfrak{M})$ is $\mathrm{p}(\mathfrak{M})$ ordered by $\in$.

Ordinal Parameters

- For technical purposes, we focus on the subclass of GSeqAs with ordinal parameters, called *generalised sequential algorithm with parameters* (*GSeqAP*s).
- Every GSeqAP $\mathfrak{M}$ has an associated finite set $\mathrm{p}(\mathfrak{M}) \subset \kappa(\mathfrak{M})$.
- Every state of a GSeqAP $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in, \bar{\mathrm{p}}(\mathfrak{M}))$ over $\sigma(\mathfrak{M})$, where $\bar{\mathrm{p}}(\mathfrak{M})$ is $\mathrm{p}(\mathfrak{M})$ ordered by $\in$.

Ordinal Parameters

- For technical purposes, we focus on the subclass of GSeqAs with ordinal parameters, called *generalised sequential algorithm with parameters* (*GSeqAP*s).
- Every GSeqAP $\mathfrak{M}$ has an associated finite set $\mathrm{p}(\mathfrak{M}) \subset \kappa(\mathfrak{M})$.
- Every state of a GSeqAP $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in, \bar{\mathrm{p}}(\mathfrak{M}))$ over $\sigma(\mathfrak{M})$, where $\bar{\mathrm{p}}(\mathfrak{M})$ is $\mathrm{p}(\mathfrak{M})$ ordered by $\in$.

Ordinal Parameters

- For technical purposes, we focus on the subclass of GSeqAs with ordinal parameters, called *generalised sequential algorithm with parameters* (*GSeqAP*s).
- Every GSeqAP $\mathfrak{M}$ has an associated finite set $\mathrm{p}(\mathfrak{M}) \subset \kappa(\mathfrak{M})$.
- Every state of a GSeqAP $\mathfrak{M}$ is an expansion of the structure $(\kappa(\mathfrak{M}); \in, \bar{\mathrm{p}}(\mathfrak{M}))$ over $\sigma(\mathfrak{M})$, where $\bar{\mathrm{p}}(\mathfrak{M})$ is $\mathrm{p}(\mathfrak{M})$ ordered by $\in$.

Input/Output Paradigm

- The interpretation of "computing B from A" is implicit in a sequential algorithm.
- We strive to make it explicit.
- Every state interprets unary relation symbols In and Out, representing the input and output tapes respectively.
- We take $\in$, In, Out and the ordinal parameters in $\mathrm{p}(\mathfrak{M})$ to be primitives, whereas other symbols in $\sigma(\mathfrak{M})$ are considered *declared variables* in the programming sense.

Input/Output Paradigm

- The interpretation of "computing B from A" is implicit in a sequential algorithm.
- We strive to make it explicit.
- Every state interprets unary relation symbols In and Out, representing the input and output tapes respectively.
- We take $\in$, In, Out and the ordinal parameters in $\mathrm{p}(\mathfrak{M})$ to be primitives, whereas other symbols in $\sigma(\mathfrak{M})$ are considered *declared variables* in the programming sense.

Input/Output Paradigm

- The interpretation of "computing B from A" is implicit in a sequential algorithm.
- We strive to make it explicit.
- Every state interprets unary relation symbols In and Out, representing the input and output tapes respectively.
- We take $\in$, In, Out and the ordinal parameters in $\mathrm{p}(\mathfrak{M})$ to be primitives, whereas other symbols in $\sigma(\mathfrak{M})$ are considered *declared variables* in the programming sense.

Input/Output Paradigm

- The interpretation of "computing B from A" is implicit in a sequential algorithm.
- We strive to make it explicit.
- Every state interprets unary relation symbols In and Out, representing the input and output tapes respectively.
- We take $\in$, In, Out and the ordinal parameters in $\mathrm{p}(\mathfrak{M})$ to be primitives, whereas other symbols in $\sigma(\mathfrak{M})$ are considered *declared variables* in the programming sense.

Input/Output Paradigm

- The interpretation of "computing B from A" is implicit in a sequential algorithm.
- We strive to make it explicit.
- Every state interprets unary relation symbols In and Out, representing the input and output tapes respectively.
- We take $\in$, In, Out and the ordinal parameters in $\mathrm{p}(\mathfrak{M})$ to be primitives, whereas other symbols in $\sigma(\mathfrak{M})$ are considered *declared variables* in the programming sense.

Transition Formulas

- Let $\mathfrak{M}$ be a GSeqAP.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\phi^{\ulcorner X \urcorner}$ of some specific form over $\sigma(\mathfrak{M})$.
- $\phi^{\ulcorner X \urcorner}$ specifies "pointwise" the next state's interpretation of $\ulcorner X \urcorner$ based on the current state's interpretation.
- The transition formula $\phi^{\tau}_{\mathfrak{M}}$ of $\mathfrak{M}$ is then defined to be the conjunction of the $\phi^{\ulcorner X \urcorner}$s.
- This definition obeys a parallelised, infinitary version of Sieg's principles.

Transition Formulas

- Let $\mathfrak{M}$ be a GSeqAP.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\phi^{\ulcorner X \urcorner}$ of some specific form over $\sigma(\mathfrak{M})$.
- $\phi^{\ulcorner X \urcorner}$ specifies "pointwise" the next state's interpretation of $\ulcorner X \urcorner$ based on the current state's interpretation.
- The transition formula $\phi^{\tau}_{\mathfrak{M}}$ of $\mathfrak{M}$ is then defined to be the conjunction of the $\phi^{\ulcorner X \urcorner}$s.
- This definition obeys a parallelised, infinitary version of Sieg's principles.

Transition Formulas

- Let $\mathfrak{M}$ be a GSeqAP.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\phi^{\ulcorner X \urcorner}$ of some specific form over $\sigma(\mathfrak{M})$.
- $\phi^{\ulcorner X \urcorner}$ specifies "pointwise" the next state's interpretation of $\ulcorner X \urcorner$ based on the current state's interpretation.
- The transition formula $\phi^{\tau}_{\mathfrak{M}}$ of $\mathfrak{M}$ is then defined to be the conjunction of the $\phi^{\ulcorner X \urcorner}$s.
- This definition obeys a parallelised, infinitary version of Sieg's principles.

Transition Formulas

- Let $\mathfrak{M}$ be a GSeqAP.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\phi^{\ulcorner X \urcorner}$ of some specific form over $\sigma(\mathfrak{M})$.
- $\phi^{\ulcorner X \urcorner}$ specifies "pointwise" the next state's interpretation of $\ulcorner X \urcorner$ based on the current state's interpretation.
- The transition formula $\phi^{\tau}_{\mathfrak{M}}$ of $\mathfrak{M}$ is then defined to be the conjunction of the $\phi^{\ulcorner X \urcorner}$s.
- This definition obeys a parallelised, infinitary version of Sieg's principles.

Transition Formulas

- Let $\mathfrak{M}$ be a GSeqAP.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\phi^{\ulcorner X \urcorner}$ of some specific form over $\sigma(\mathfrak{M})$.
- $\phi^{\ulcorner X \urcorner}$ specifies "pointwise" the next state's interpretation of $\ulcorner X \urcorner$ based on the current state's interpretation.
- The transition formula $\phi^{\tau}_{\mathfrak{M}}$ of $\mathfrak{M}$ is then defined to be the conjunction of the $\phi^{\ulcorner X \urcorner}$s.
- This definition obeys a parallelised, infinitary version of Sieg's principles.

Transition Formulas

- Let $\mathfrak{M}$ be a GSeqAP.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\phi^{\ulcorner X \urcorner}$ of some specific form over $\sigma(\mathfrak{M})$.
- $\phi^{\ulcorner X \urcorner}$ specifies "pointwise" the next state's interpretation of $\ulcorner X \urcorner$ based on the current state's interpretation.
- The transition formula $\phi^{\tau}_{\mathfrak{M}}$ of $\mathfrak{M}$ is then defined to be the conjunction of the $\phi^{\ulcorner X \urcorner}$s.
- This definition obeys a parallelised, infinitary version of Sieg's principles.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

Initial States

- We are left we defining what it means to be an initial state of a GSeqAP $\mathfrak{M}$.
- For each declared variable $\ulcorner X \urcorner \in \sigma(\mathfrak{M})$, we associate it with a Δ_0 formula $\psi^{\ulcorner X \urcorner}$ of some specific form over $\in$.
- $\psi^{\ulcorner X \urcorner}$ ought to specify "pointwise" the unique starting behaviour of $\ulcorner X \urcorner$.
- Then $\phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$, defined to be the conjunction of the $\psi^{\ulcorner X \urcorner}$s, determines the starting behaviour of all declared variables.
- If $A \subset \kappa(\mathfrak{M})$, then there is a unique initial state s of $\mathfrak{M}$ such that s interprets In as A, Out as $\emptyset$ and $s \models \phi^d_{\mathfrak{M}}$ of $\mathfrak{M}$.
- These are exactly all the initial states.

GSeqAP: Definition

- A GSeqAP $\mathfrak{M}$ is a tuple

$$(\kappa(\mathfrak{M}), \sigma(\mathfrak{M}), \mathrm{p}(\mathfrak{M}), \phi^{\tau}_{\mathfrak{M}}, \phi^{d}_{\mathfrak{M}})$$

  satisfying the properties we have mentioned.
- Let $S(\mathfrak{M})$ denote the set of states of $\mathfrak{M}$, derivable from $\kappa(\mathfrak{M})$, $\sigma(\mathfrak{M})$ and $\mathrm{p}(\mathfrak{M})$.
- The transition (partial) function $\tau_{\mathfrak{M}}$ of $\mathfrak{M}$ can then be derived from $S(\mathfrak{M})$ and $\phi^{\tau}_{\mathfrak{M}}$ as follows:

$$\begin{aligned}\tau_{\mathfrak{M}} := \{(s_0, s_1) \in S(\mathfrak{M})^2 : \exists! x \in S(\mathfrak{M}) \ ((s_0, x) \models_2 \phi^{\tau}_{\mathfrak{M}}) \\ \wedge (s_0, s_1) \models_2 \phi^{\tau}_{\mathfrak{M}}\}.\end{aligned}$$

- $S(\mathfrak{M})$ and $\tau_{\mathfrak{M}}$ are absolute for transitive models of (any sufficiently strong) set theory.

GSeqAP: Definition

- A GSeqAP $\mathfrak{M}$ is a tuple

$$(\kappa(\mathfrak{M}), \sigma(\mathfrak{M}), \mathrm{p}(\mathfrak{M}), \phi^{\tau}_{\mathfrak{M}}, \phi^{d}_{\mathfrak{M}})$$

  satisfying the properties we have mentioned.
- Let $S(\mathfrak{M})$ denote the set of states of $\mathfrak{M}$, derivable from $\kappa(\mathfrak{M})$, $\sigma(\mathfrak{M})$ and $\mathrm{p}(\mathfrak{M})$.
- The transition (partial) function $\tau_{\mathfrak{M}}$ of $\mathfrak{M}$ can then be derived from $S(\mathfrak{M})$ and $\phi^{\tau}_{\mathfrak{M}}$ as follows:

$$\begin{aligned} \tau_{\mathfrak{M}} := \{(s_0, s_1) \in S(\mathfrak{M})^2 : \exists! x \in S(\mathfrak{M}) \ ((s_0, x) \models_2 \phi^{\tau}_{\mathfrak{M}}) \\ \wedge (s_0, s_1) \models_2 \phi^{\tau}_{\mathfrak{M}}\}. \end{aligned}$$

- $S(\mathfrak{M})$ and $\tau_{\mathfrak{M}}$ are absolute for transitive models of (any sufficiently strong) set theory.

GSeqAP: Definition

- A GSeqAP $\mathfrak{M}$ is a tuple

$$(\kappa(\mathfrak{M}), \sigma(\mathfrak{M}), \mathrm{p}(\mathfrak{M}), \phi^{\tau}_{\mathfrak{M}}, \phi^{d}_{\mathfrak{M}})$$

  satisfying the properties we have mentioned.

- Let $S(\mathfrak{M})$ denote the set of states of $\mathfrak{M}$, derivable from $\kappa(\mathfrak{M})$, $\sigma(\mathfrak{M})$ and $\mathrm{p}(\mathfrak{M})$.
- The transition (partial) function $\tau_{\mathfrak{M}}$ of $\mathfrak{M}$ can then be derived from $S(\mathfrak{M})$ and $\phi^{\tau}_{\mathfrak{M}}$ as follows:

$$\begin{aligned}\tau_{\mathfrak{M}} := \{(s_0, s_1) \in S(\mathfrak{M})^2 : \exists! x \in S(\mathfrak{M}) \ ((s_0, x) \models_2 \phi^{\tau}_{\mathfrak{M}}) \\ \wedge (s_0, s_1) \models_2 \phi^{\tau}_{\mathfrak{M}}\}.\end{aligned}$$

- $S(\mathfrak{M})$ and $\tau_{\mathfrak{M}}$ are absolute for transitive models of (any sufficiently strong) set theory.

GSeqAP: Definition

- A GSeqAP $\mathfrak{M}$ is a tuple

$$(\kappa(\mathfrak{M}), \sigma(\mathfrak{M}), \mathrm{p}(\mathfrak{M}), \phi^{\tau}_{\mathfrak{M}}, \phi^{d}_{\mathfrak{M}})$$

  satisfying the properties we have mentioned.
- Let $S(\mathfrak{M})$ denote the set of states of $\mathfrak{M}$, derivable from $\kappa(\mathfrak{M})$, $\sigma(\mathfrak{M})$ and $\mathrm{p}(\mathfrak{M})$.
- The transition (partial) function $\tau_{\mathfrak{M}}$ of $\mathfrak{M}$ can then be derived from $S(\mathfrak{M})$ and $\phi^{\tau}_{\mathfrak{M}}$ as follows:

$$\tau_{\mathfrak{M}} := \{(s_0, s_1) \in S(\mathfrak{M})^2 : \exists! x \in S(\mathfrak{M})\ ((s_0, x) \models_2 \phi^{\tau}_{\mathfrak{M}}) \\ \wedge (s_0, s_1) \models_2 \phi^{\tau}_{\mathfrak{M}}\}.$$

- $S(\mathfrak{M})$ and $\tau_{\mathfrak{M}}$ are absolute for transitive models of (any sufficiently strong) set theory.

GSeqAP: Definition

- A GSeqAP $\mathfrak{M}$ is a tuple

$$(\kappa(\mathfrak{M}), \sigma(\mathfrak{M}), \mathrm{p}(\mathfrak{M}), \phi^{\tau}_{\mathfrak{M}}, \phi^{d}_{\mathfrak{M}})$$

  satisfying the properties we have mentioned.
- Let $S(\mathfrak{M})$ denote the set of states of $\mathfrak{M}$, derivable from $\kappa(\mathfrak{M})$, $\sigma(\mathfrak{M})$ and $\mathrm{p}(\mathfrak{M})$.
- The transition (partial) function $\tau_{\mathfrak{M}}$ of $\mathfrak{M}$ can then be derived from $S(\mathfrak{M})$ and $\phi^{\tau}_{\mathfrak{M}}$ as follows:

$$\begin{aligned}\tau_{\mathfrak{M}} := \{(s_0, s_1) \in S(\mathfrak{M})^2 : \exists! x \in S(\mathfrak{M}) \ ((s_0, x) \models_2 \phi^{\tau}_{\mathfrak{M}}) \\ \wedge (s_0, s_1) \models_2 \phi^{\tau}_{\mathfrak{M}}\}.\end{aligned}$$

- $S(\mathfrak{M})$ and $\tau_{\mathfrak{M}}$ are absolute for transitive models of (any sufficiently strong) set theory.

GSeqAP: Definition

- A GSeqAP $\mathfrak{M}$ is a tuple

$$(\kappa(\mathfrak{M}), \sigma(\mathfrak{M}), \mathrm{p}(\mathfrak{M}), \phi^{\tau}_{\mathfrak{M}}, \phi^{d}_{\mathfrak{M}})$$

  satisfying the properties we have mentioned.
- Let $S(\mathfrak{M})$ denote the set of states of $\mathfrak{M}$, derivable from $\kappa(\mathfrak{M})$, $\sigma(\mathfrak{M})$ and $\mathrm{p}(\mathfrak{M})$.
- The transition (partial) function $\tau_{\mathfrak{M}}$ of $\mathfrak{M}$ can then be derived from $S(\mathfrak{M})$ and $\phi^{\tau}_{\mathfrak{M}}$ as follows:

$$\begin{aligned}\tau_{\mathfrak{M}} := \{(s_0, s_1) \in S(\mathfrak{M})^2 : \exists! x \in S(\mathfrak{M})\ ((s_0, x) \models_2 \phi^{\tau}_{\mathfrak{M}})\\ \wedge (s_0, s_1) \models_2 \phi^{\tau}_{\mathfrak{M}}\}.\end{aligned}$$

- $S(\mathfrak{M})$ and $\tau_{\mathfrak{M}}$ are absolute for transitive models of (any sufficiently strong) set theory.

Outline

A Call for Transfinite Runs

- *Runs* (or *computations*) are presumed to be finite in a sequential algorithm.
- Even with our restrictions, one can easily compress a finite run into a single step.
- If we allow transfinite inputs and outputs, it makes sense to allow transfinite runs.
- The transition formula dictates what happens at successor time-steps; what about limit time-steps?

A Call for Transfinite Runs

- *Runs* (or *computations*) are presumed to be finite in a sequential algorithm.
- Even with our restrictions, one can easily compress a finite run into a single step.
- If we allow transfinite inputs and outputs, it makes sense to allow transfinite runs.
- The transition formula dictates what happens at successor time-steps; what about limit time-steps?

A Call for Transfinite Runs

- *Runs* (or *computations*) are presumed to be finite in a sequential algorithm.
- Even with our restrictions, one can easily compress a finite run into a single step.
- If we allow transfinite inputs and outputs, it makes sense to allow transfinite runs.
- The transition formula dictates what happens at successor time-steps; what about limit time-steps?

A Call for Transfinite Runs

- *Runs* (or *computations*) are presumed to be finite in a sequential algorithm.
- Even with our restrictions, one can easily compress a finite run into a single step.
- If we allow transfinite inputs and outputs, it makes sense to allow transfinite runs.
- The transition formula dictates what happens at successor time-steps; what about limit time-steps?

A Call for Transfinite Runs

- *Runs* (or *computations*) are presumed to be finite in a sequential algorithm.
- Even with our restrictions, one can easily compress a finite run into a single step.
- If we allow transfinite inputs and outputs, it makes sense to allow transfinite runs.
- The transition formula dictates what happens at successor time-steps; what about limit time-steps?

Taking Limits

- We want the definition of a limit state to depend locally on the states that come before it in a run.
- This can again be formalised using the first-order truth predicate in a "pointwise" Δ_0 way.
- Under this constraint, we show that with enough space (increasing $\kappa(\mathfrak{M})$ if necessary), taking limit/limsup/liminf wherever possible "pointwise" can simulate all other definitions of a limit state.

Taking Limits

- We want the definition of a limit state to depend locally on the states that come before it in a run.
- This can again be formalised using the first-order truth predicate in a "pointwise" Δ_0 way.
- Under this constraint, we show that with enough space (increasing $\kappa(\mathfrak{M})$ if necessary), taking limit/limsup/liminf wherever possible "pointwise" can simulate all other definitions of a limit state.

Taking Limits

- We want the definition of a limit state to depend locally on the states that come before it in a run.
- This can again be formalised using the first-order truth predicate in a "pointwise" Δ_0 way.
- Under this constraint, we show that with enough space (increasing $\kappa(\mathfrak{M})$ if necessary), taking limit/limsup/liminf wherever possible "pointwise" can simulate all other definitions of a limit state.

Taking Limits

- We want the definition of a limit state to depend locally on the states that come before it in a run.
- This can again be formalised using the first-order truth predicate in a "pointwise" Δ_0 way.
- Under this constraint, we show that with enough space (increasing $\kappa(\mathfrak{M})$ if necessary), taking limit/limsup/liminf wherever possible "pointwise" can simulate all other definitions of a limit state.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \restriction \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \restriction \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \restriction \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \upharpoonright \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \restriction \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \restriction \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Terminating Runs

Definition

A *terminating run* of a GSeqAP $\mathfrak{M}$ is a function from some successor ordinal δ into $S(\mathfrak{M})$ such that

- $Sq(0)$ is an initial state,
- $Sq(\alpha+1) = \tau_{\mathfrak{M}}(Sq(\alpha)) \neq Sq(\alpha)$ for all $\alpha < \delta - 1$,
- $Sq(\gamma)$ is obtained from $Sq \restriction \gamma$ "pointwise" by taking liminf if possible and resetting to 0 otherwise, for limit ordinals $\gamma < \delta$, and
- $\tau_{\mathfrak{M}}(Sq(\delta-1)) = Sq(\delta-1)$.

We call δ the *length* of Sq.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \; R \; \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \mathrel{R} \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \; R \; \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \; R \; \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
    - R is transitive,
    - R sidesteps the need for an oracle-analogue, and
    - a set x being *R-computable* means $x \; R \; \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \mathrel{R} \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq_{\kappa}^{P} A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq_{\kappa}^{P,s} A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^{P} A$ iff $B \leq_{\kappa}^{P} A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \; R \; \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq_{\kappa}^{P} A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq_{\kappa}^{P,s} A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^{P} A$ iff $B \leq_{\kappa}^{P} A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \; R \; \emptyset$.

Relative Computability Relations

- A GSeqAP computes B from A iff it has a run that starts with In interpreted as A and terminates with Out interpreted as B.
- $B \leq^P_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A.
- $B \leq^{P,s}_\kappa A$ iff some GSeqAP $\mathfrak{M}$ with $\kappa(\mathfrak{M}) = \kappa$ computes B from A in a terminating run of length $< \kappa$.
- $B \leq^P A$ iff $B \leq^P_\kappa A$ for some limit ordinal κ.
- If R is one of the above relations, then
  - R is transitive,
  - R sidesteps the need for an oracle-analogue, and
  - a set x being *R-computable* means $x \; R \; \emptyset$.

Degrees of Constructibility

- We show that $B \leq^P A$ iff $B \in L[A]$, where $L[A]$ is Gödel's constructible universe relative to A.
- This means the degrees of relative computability derived from $\leq^P$ are exactly the degrees of constructibility.
- The very restricted (at first glance) programming we allow in GSeqAPs actually has the full power of transfinite recursion.
- In fact, the above is already true for the much lower-level α-machines in Koepke's theory of α-computability (see e.g. [7]).

Degrees of Constructibility

- We show that $B \leq^P A$ iff $B \in L[A]$, where $L[A]$ is Gödel's constructible universe relative to A.
- This means the degrees of relative computability derived from $\leq^P$ are exactly the degrees of constructibility.
- The very restricted (at first glance) programming we allow in GSeqAPs actually has the full power of transfinite recursion.
- In fact, the above is already true for the much lower-level α-machines in Koepke's theory of α-computability (see e.g. [7]).

Degrees of Constructibility

- We show that $B \leq^P A$ iff $B \in L[A]$, where $L[A]$ is Gödel's constructible universe relative to A.
- This means the degrees of relative computability derived from $\leq^P$ are exactly the degrees of constructibility.
- The very restricted (at first glance) programming we allow in GSeqAPs actually has the full power of transfinite recursion.
- In fact, the above is already true for the much lower-level α-machines in Koepke's theory of α-computability (see e.g. [7]).

Degrees of Constructibility

- We show that $B \leq^P A$ iff $B \in L[A]$, where $L[A]$ is Gödel's constructible universe relative to A.
- This means the degrees of relative computability derived from $\leq^P$ are exactly the degrees of constructibility.
- The very restricted (at first glance) programming we allow in GSeqAPs actually has the full power of transfinite recursion.
- In fact, the above is already true for the much lower-level α-machines in Koepke's theory of α-computability (see e.g. [7]).

Degrees of Constructibility

- We show that $B \leq^P A$ iff $B \in L[A]$, where $L[A]$ is Gödel's constructible universe relative to A.
- This means the degrees of relative computability derived from $\leq^P$ are exactly the degrees of constructibility.
- The very restricted (at first glance) programming we allow in GSeqAPs actually has the full power of transfinite recursion.
- In fact, the above is already true for the much lower-level α-machines in Koepke's theory of α-computability (see e.g. [7]).

The Relations $\leq_{\kappa}^{P,s}$ and $\leq_{\kappa}^{P}$

- An ordinal κ is *admissible* iff $(L_{\kappa}; \in)$ is a model of Kripke-Platek set theory.
- When κ is admissible, $\leq_{\kappa}^{P,s}$ and $\leq_{\kappa}^{P}$ are both defined.
- It can be proven that if κ_1 and κ_2 are admissible, $\kappa_1 < \kappa_2$ and $B \leq_{\kappa_1}^{P} A$ (resp. $B \leq_{\kappa_1}^{P,s} A$), then $B \leq_{\kappa_2}^{P} A$ (resp. $B \leq_{\kappa_2}^{P,s} A$).
- In this case we call $\leq_{\kappa}^{P}$ (resp. $\leq_{\kappa}^{P,s}$) *upward persistent*.

The Relations $\leq_\kappa^{P,s}$ and $\leq_\kappa^P$

- An ordinal κ is *admissible* iff $(L_\kappa; \in)$ is a model of Kripke-Platek set theory.
- When κ is admissible, $\leq_\kappa^{P,s}$ and $\leq_\kappa^P$ are both defined.
- It can be proven that if κ_1 and κ_2 are admissible, $\kappa_1 < \kappa_2$ and $B \leq_{\kappa_1}^P A$ (resp. $B \leq_{\kappa_1}^{P,s} A$), then $B \leq_{\kappa_2}^P A$ (resp. $B \leq_{\kappa_2}^{P,s} A$).
- In this case we call $\leq_\kappa^P$ (resp. $\leq_\kappa^{P,s}$) *upward persistent*.

The Relations $\leq_\kappa^{P,s}$ and $\leq_\kappa^P$

- An ordinal κ is *admissible* iff $(L_\kappa; \in)$ is a model of Kripke-Platek set theory.
- When κ is admissible, $\leq_\kappa^{P,s}$ and $\leq_\kappa^P$ are both defined.
- It can be proven that if κ_1 and κ_2 are admissible, $\kappa_1 < \kappa_2$ and $B \leq_{\kappa_1}^P A$ (resp. $B \leq_{\kappa_1}^{P,s} A$), then $B \leq_{\kappa_2}^P A$ (resp. $B \leq_{\kappa_2}^{P,s} A$).
- In this case we call $\leq_\kappa^P$ (resp. $\leq_\kappa^{P,s}$) *upward persistent*.

The Relations $\leq_\kappa^{P,s}$ and $\leq_\kappa^P$

- An ordinal κ is *admissible* iff $(L_\kappa; \in)$ is a model of Kripke-Platek set theory.
- When κ is admissible, $\leq_\kappa^{P,s}$ and $\leq_\kappa^P$ are both defined.
- It can be proven that if κ_1 and κ_2 are admissible, $\kappa_1 < \kappa_2$ and $B \leq_{\kappa_1}^P A$ (resp. $B \leq_{\kappa_1}^{P,s} A$), then $B \leq_{\kappa_2}^P A$ (resp. $B \leq_{\kappa_2}^{P,s} A$).
- In this case we call $\leq_\kappa^P$ (resp. $\leq_\kappa^{P,s}$) *upward persistent.*

The Relations $\leq_{\kappa}^{P,s}$ and $\leq_{\kappa}^{P}$

- An ordinal κ is *admissible* iff $(L_{\kappa}; \in)$ is a model of Kripke-Platek set theory.
- When κ is admissible, $\leq_{\kappa}^{P,s}$ and $\leq_{\kappa}^{P}$ are both defined.
- It can be proven that if κ_1 and κ_2 are admissible, $\kappa_1 < \kappa_2$ and $B \leq_{\kappa_1}^{P} A$ (resp. $B \leq_{\kappa_1}^{P,s} A$), then $B \leq_{\kappa_2}^{P} A$ (resp. $B \leq_{\kappa_2}^{P,s} A$).
- In this case we call $\leq_{\kappa}^{P}$ (resp. $\leq_{\kappa}^{P,s}$) *upward persistent*.

Summary Table

In the following table we compare various relative computability relations when they are restricted to subsets of admissible ordinals α.

Property \ Relation	$\leq_\alpha$	$\preceq_\alpha$	$\leq_\alpha^P$	$\leq_\alpha^{P,s}$
Oracle-analogue?	✓	✓	✗	✗
Transitive?	✓	✗	✓	✓
Upward persistent?	✗	✓	✓	✓
Appears in…	α-recursion	α-computability		

Summary Table

In the following table we compare various relative computability relations when they are restricted to subsets of admissible ordinals α.

Property \ Relation	$\leq_\alpha$	$\preceq_\alpha$	$\leq_\alpha^P$	$\leq_\alpha^{P,s}$
Oracle-analogue?	✓	✓	✗	✗
Transitive?	✓	✗	✓	✓
Upward persistent?	✗	✓	✓	✓
Appears in...	α-recursion	α-computability		

Summary Table

In the following table we compare various relative computability relations when they are restricted to subsets of admissible ordinals α.

Property \ Relation	$\leq_\alpha$	$\preceq_\alpha$	$\leq_\alpha^P$	$\leq_\alpha^{P,s}$
Oracle-analogue?	✓	✓	✗	✗
Transitive?	✓	✗	✓	✓
Upward persistent?	✗	✓	✓	✓
Appears in…	α-recursion	α-computability		

Outline

1. Introduction
   - Background: Infinitary Computation
   - Background: Formulas as Programs
   - Overview
2. Generalised Computation Within V
   - Sequential Algorithms
   - Generalised Sequential Algorithms
   - Relative Computability
3. Generalised Computation Beyond V
   - Degrees of Small Extensions
   - Local Method Definitions

Generators over V

- Going further in the direction of generalised computation, can we maintain the definition of constructibility degrees, but swop L for V?
- In essence, we want to group sets outside V based on their power as generators over V.

Generators over V

- Going further in the direction of generalised computation, can we maintain the definition of constructibility degrees, but swop L for V?
- In essence, we want to group sets outside V based on their power as generators over V.

Generators over V

- Going further in the direction of generalised computation, can we maintain the definition of constructibility degrees, but swop L for V?
- In essence, we want to group sets outside V based on their power as generators over V.

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at* V is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at* V is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of U_1* iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of U_2 over U_1*, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at V* is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Assorted Multiversal Properties

Due to Jensen's result on "coding the universe", we have the following theorem.

Theorem (Jensen)

Given a CTM V, $(\mathbf{M}_S(V), \subset)$ is a cofinal subposet of $(\mathbf{M}(V), \subset)$.

Assorted Multiversal Properties

Due to Jensen's result on "coding the universe", we have the following theorem.

Theorem (Jensen)

Given a CTM V, $(\mathbf{M}_S(V), \subset)$ is a cofinal subposet of $(\mathbf{M}(V), \subset)$.

Assorted Multiversal Properties

Due to Jensen's result on "coding the universe", we have the following theorem.

Theorem (Jensen)

Given a CTM V, $(\mathbf{M}_S(V), \subset)$ is a cofinal subposet of $(\mathbf{M}(V), \subset)$.

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G}/\equiv^S, \leq^S/\equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G}/\equiv^S, \leq^S/\equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G}/\equiv^S, \leq^S/\equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G}/\equiv^S, \leq^S/\equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G}/\equiv^S, \leq^S/\equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G}/\equiv^S, \leq^S/\equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

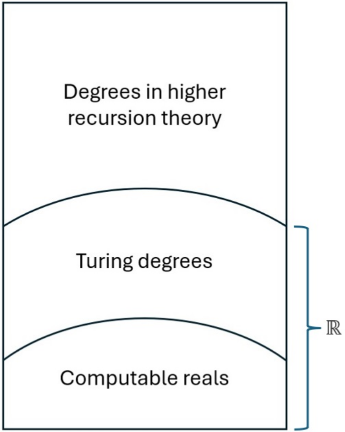
- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G}/\equiv^S, \leq^S/\equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G}/\equiv^S, \leq^S/\equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Drawing Parallels

Drawing Parallels



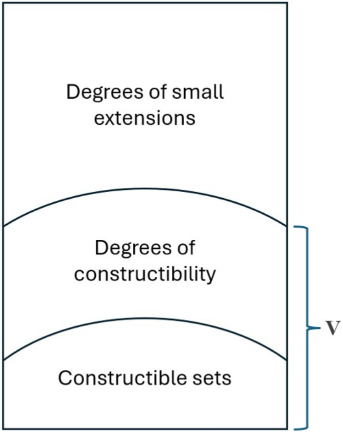


Figure: comparison between conventional notions of relative computability (left) and our generalised notions (right).

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ *is downward-closed in* $\mathbf{M}(V)$*, and thus also in* $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ *is a cofinal subposet of* $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ *is downward-closed in* $\mathbf{M}(V)$*, and thus also in* $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ *is a cofinal subposet of* $(\mathbf{M}_S(V), \subset)$.

Accessibility of Forcing

- The previous two facts tell us there are many objects inaccessible by forcing.
- Do these objects have "local first-order properties" not shared by any set in any forcing extension?

Accessibility of Forcing

- The previous two facts tell us there are many objects inaccessible by forcing.
- Do these objects have "local first-order properties" not shared by any set in any forcing extension?

Accessibility of Forcing

- The previous two facts tell us there are many objects inaccessible by forcing.
- Do these objects have "local first-order properties" not shared by any set in any forcing extension?

Outline

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (“}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{”)} \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (“}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{”)} \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (``}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{'')} \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists\mathfrak{M}\ (\text{“}\mathfrak{M} \text{ is a model of } \mathscr{T}\text{”}) \iff \mathscr{T} \text{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (``}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{'')} \iff \mathscr{T} \textit{ is consistent.}$$

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

> "there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

> "there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \text{Eval}^V(F(\mathscr{T})) \subset \text{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

> "there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Comparing Local Methods

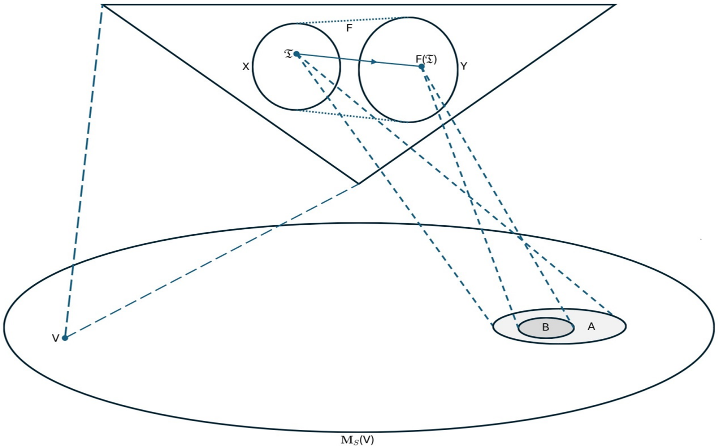


Figure: Visual representation of a function F witnessing $X \leq^M Y$, where X and Y are local method definitions.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\Sigma^{\mathsf{M}}_{\mathsf{n}}$ and $\Pi^{\mathsf{M}}_{\mathsf{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\Sigma^{\mathrm{M}}_{\mathrm{n}}$ and $\Pi^{\mathrm{M}}_{\mathrm{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\Sigma^{\mathrm{M}}_{\mathrm{n}}$ and $\Pi^{\mathrm{M}}_{\mathrm{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\mathsf{\Sigma}^{\mathsf{M}}_{\mathsf{n}}$ and $\mathsf{\Pi}^{\mathsf{M}}_{\mathsf{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^{V}(\mathscr{T}) = \mathrm{Eval}^{V}(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^{M} \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all* $1 \leq n < \omega$.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all* $1 \leq n < \omega$.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all* $1 \leq n < \omega$.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all $1 \leq n < \omega$.*

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, Fg $\leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, $\mathrm{Fg} \leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, $\mathrm{Fg} \leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, Fg $\leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, Fg $\leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathrm{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathrm{Fg}$. $\square$

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathsf{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$. $\square$

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathsf{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$. $\square$

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathsf{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$. $\square$

Further Results

- We can refine the function witnessing $\Pi_2^{\mathrm{M}} \leq^M \mathrm{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Further Results

- We can refine the function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Further Results

- We can refine the function witnessing $\Pi_2^{\mathrm{M}} \leq^M \mathrm{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Further Results

- We can refine the function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
    - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathsf{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \neq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathsf{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \neq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathrm{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathsf{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathrm{Fg}$?

References I

[1] Gaisi Takeuti. "On the recursive functions of ordinal numbers". In: *Journal of the Mathematical Society of Japan* 12.2 (1960), pp. 119–128.

[2] Georg Kreisel and Gerald E Sacks. "Metarecursive sets". In: *The Journal of Symbolic Logic* 30.3 (1965), pp. 318–338.

[3] Dag Normann. "Set recursion". In: *Studies in Logic and the Foundations of Mathematics*. Vol. 94. Elsevier, 1978, pp. 303–320.

[4] Kurt Gödel. *Collected Works, Volume 2: Publications 1938-1974*. Oxford, England and New York, NY, USA, 1986.

References II

[5] Lenore Blum, Mike Shub, and Steve Smale. "On a theory of computation and complexity over the real numbers: NP-completeness, recursive functions and universal machines". In: *Bulletin of the American Mathematical Society* 21.1 (1989), pp. 1–46.

[6] Joel David Hamkins and Andy Lewis. "Infinite time Turing machines". In: *The Journal of Symbolic Logic* 65.2 (2000), pp. 567–604.

[7] Peter Koepke and Martin Koerwien. "Ordinal computations". In: *Mathematical Structures in Computer Science* 16.5 (2006), pp. 867–884.

[8] Wilfried Sieg. "Gödel on computability". In: *Philosophia Mathematica* 14.2 (2006), pp. 189–207.

References III

[9] Keng Meng Ng, Nazanin R Tavana, and Yue Yang. “A recursion theoretic foundation of computation over real numbers”. In: *Journal of Logic and Computation* 31.7 (July 2021), pp. 1660–1689. ISSN: 0955-792X. DOI: 10.1093/logcom/exab042. eprint: https://academic.oup.com/logcom/article-pdf/31/7/1660/40820416/exab042.pdf. URL: https://doi.org/10.1093/logcom/exab042.

Thank You!